

花卉小助手

初识卷积神经网络

一张RGB图片的三通道彩色图片，先通过卷积把图片求图片局部的特征，让神经元能“看到”图片的局部，这时的特征值就是图片某个局部的特征值，再通过池化把局部与局部合并，使神经元能“看到”更大的图片的部分，这时神经元里的特征值就能代表更大的图片的部分，一直不断地进行池化操作扩大神经元的感受野，直到神经元能“看到”整张图片，这样我们就把这样一张图片由若干数字特征表示了出来，再通过这些数字特征去做神经网络分类。

张量直观的看来就是一个多维数组，pytorch上的tensor支持自动求导，一维时可以将其看作向量，二维时可以将其看作矩阵，而整个卷积层的权重则是四维张量（单个卷积核的权重是三维张量，维度分别为长、宽、输入通道）。

一个卷积神经网络一般分为卷积层（初步把RGB图像转化为数字）、激活层（归一化、引入非线性）、池化层（增大感受野）、（前三个循环）、全局平均池化、（随机dropout正则化+线性全连接）输出层。

CNN中的卷积和数学上的卷积感觉上完全不一样，数学上的卷积是形如 $(f * g)(t) = \int f(\tau)g(t - \tau)$ 的形式，式子中两个函数的参数有翻转的操作，而CNN中提到的卷积我感觉其实就是每个输出通道对应一个卷积核，用这个卷积核对一个局部的某个输入通道的所有的值加权求和。卷积核可以理解为一个权重集合，用于对不同的输入通道的局部矩形里的输入值进行加权求和，步长则用于度量每次打算把这个局部矩形的位置在整张图中向下移动多少。

常见的池化有有maxpool（取最大值，目的是保留局部图像最突出的特征）、AvgPool（取平均值，更能反映图像的整体信息）和AdaptiveAvgPool2d((1,1))（全局平均化，概括一整张图）。一个神经元的感受野是能影响到他的取值的原始图像范围，可以理解为他能“看到”的图像的范围。

激活层分为两个函数：BatchNorm2d 和 ReLU，BatchNorm2d是把数据归一化，使训练更稳定，ReLU是为了引入非线性，并且防止梯度消失。最终卷积神经网络输出的是每种类别的logits，由softmax操作转变为概率，且所有类别的概率和为1。

数据预处理

分别遍历data文件夹下的train目录和test目录，把搜到的第一个子目录标记为类别并重新在这个目录下面搜索图片。

对于每张图片的路径，先考虑处理数据损坏的问题，用try进行img.verify()和img.load()测试图片是否有效，再对有效的图片进行hash去重，最终把去重后的目录列表传回。另外，在处理train文件夹时，会把训练集中图片的九分之一出去给验证集。

注：在去重时发现有一模一样的图片在两种不同类别的文件夹里，按找去重规则会去掉后被检索到的类别的那张。

接下来对划分后的训练集的图片转化为张量求平均数和标准差，为后续标准化图像做准备。

最后把训练集、验证集、测试集以及平均数和标准差存到一个json文件里方便dataset读取。

在dataset中，读取datadeal得到的json文件，创建一个类方便dataloader调用，并封装好针对训练集和针对验证集、测试集的数据处理，不管是训练集、验证集还是测试集都要进行resize（224*224）、转化为张量和标准化（为防止数据泄露用的是训练集的平均数和标准差），而训练集的数据还要额外进行数据增强，把数据增强

封装在数据处理里，就不用真的把数据增强后的图片存储起来，只需要在每次dataloader调用时临时增强即可。使用以下几种增强方法：

1.RandomResizedCrop，随机裁剪图像的一块区域再进行缩放。

2.RandomHorizontalFlip，以一定概率对图片进行左右翻转

3.ColorJitter，随机调整图像的亮度、对比度、饱和度和色调，在这里不能调整色调，因为色调是识别花卉的重要因素，调整很可能会引发误判。

4.RandomErasing，随机选一块小的矩形区域擦除，注意这项要在数据转化为张量后进行。

炼丹

超参数在机器学习中是人为约定的、不是机器学来的参数，用于调控机器学习的进程，超参数有训练轮数EPOCH、学习率lr、每批次数据量BATCH_SIZE等。这些参数由人为设定，并根据代码跑出来的结果进行调整。

优化器用于根据得到的梯度更新参数，不同的优化器更新参数的方式、幅度不同，常见的优化器有SGD、Adam、AdamW等，这里我们选择AdamW，另外，调度器是随epoch更新学习率的工具，这里选用余弦退火。

dataset简单来说是一个自定义的数据集，负责把数据整合处理起来方便调用，而dataloader则是一个调用数据的工具，按需求从dataset那里调用数据，

结果

```
(d21) pigeon@LAPTOP-GARN0974:~/ml-week/flower_identification/flower_identification$ python src/train.py
epoch 1 | train 32.42% loss 3.1857 | val 37.14% loss 1.3393
epoch 2 | train 37.72% loss 1.3898 | val 38.20% loss 1.2975
epoch 3 | train 37.78% loss 1.3754 | val 37.67% loss 1.3396
epoch 4 | train 37.52% loss 1.3588 | val 39.52% loss 1.3054
epoch 5 | train 38.47% loss 1.3666 | val 42.44% loss 1.2769
epoch 6 | train 37.26% loss 1.3810 | val 34.22% loss 1.3129
epoch 7 | train 37.72% loss 1.3621 | val 51.19% loss 1.2576
epoch 8 | train 40.45% loss 1.3190 | val 46.95% loss 1.2170
epoch 9 | train 38.82% loss 1.3547 | val 47.75% loss 1.1813
epoch 10 | train 40.32% loss 1.2972 | val 48.28% loss 1.1648
epoch 11 | train 43.31% loss 1.2835 | val 40.32% loss 1.2305
epoch 12 | train 43.53% loss 1.2913 | val 48.54% loss 1.1831
epoch 13 | train 44.41% loss 1.2507 | val 49.87% loss 1.2303
epoch 14 | train 43.79% loss 1.2522 | val 46.15% loss 1.1651
epoch 15 | train 45.19% loss 1.2580 | val 52.25% loss 1.1396
epoch 16 | train 46.43% loss 1.2128 | val 51.99% loss 1.1601
epoch 17 | train 44.12% loss 1.2256 | val 55.70% loss 1.0464
epoch 18 | train 47.47% loss 1.2033 | val 55.70% loss 1.1303
epoch 19 | train 48.25% loss 1.1730 | val 54.38% loss 1.0662
epoch 20 | train 49.16% loss 1.1523 | val 57.29% loss 1.0401
epoch 21 | train 49.71% loss 1.1427 | val 54.11% loss 1.0975
epoch 22 | train 50.58% loss 1.1367 | val 56.50% loss 1.1292
epoch 23 | train 50.45% loss 1.1522 | val 55.70% loss 1.0845
epoch 24 | train 51.10% loss 1.1249 | val 55.70% loss 1.0312
epoch 25 | train 50.78% loss 1.1231 | val 59.42% loss 1.0125
```

epoch 26		train 51.69%	loss 1.1266		val 49.34%	loss 1.1623
epoch 27		train 51.72%	loss 1.0895		val 55.97%	loss 1.0846
epoch 28		train 52.76%	loss 1.1037		val 55.44%	loss 1.0800
epoch 29		train 51.53%	loss 1.1108		val 60.21%	loss 1.0200
epoch 30		train 52.50%	loss 1.1033		val 55.70%	loss 1.0333
epoch 31		train 53.80%	loss 1.0714		val 53.32%	loss 1.0780
epoch 32		train 54.19%	loss 1.0785		val 57.82%	loss 1.0249
epoch 33		train 54.97%	loss 1.0760		val 63.93%	loss 0.9692
epoch 34		train 55.85%	loss 1.0602		val 63.13%	loss 1.0641
epoch 35		train 57.86%	loss 1.0382		val 65.78%	loss 0.9573
epoch 36		train 58.32%	loss 1.0386		val 65.78%	loss 1.0142
epoch 37		train 58.58%	loss 1.0534		val 65.25%	loss 0.9600
epoch 38		train 59.19%	loss 1.0220		val 63.93%	loss 0.9157
epoch 39		train 61.14%	loss 0.9789		val 72.15%	loss 0.9025
epoch 40		train 63.09%	loss 0.9697		val 61.54%	loss 1.0001
epoch 41		train 64.59%	loss 0.9482		val 68.44%	loss 0.9112
epoch 42		train 65.40%	loss 0.9486		val 68.97%	loss 0.8775
epoch 43		train 64.91%	loss 0.9475		val 70.82%	loss 0.8748
epoch 44		train 65.72%	loss 0.9277		val 68.70%	loss 0.8876
epoch 45		train 68.32%	loss 0.8959		val 68.44%	loss 0.8204
epoch 46		train 66.31%	loss 0.9097		val 67.90%	loss 0.8822
epoch 47		train 70.11%	loss 0.8602		val 75.60%	loss 0.8297
epoch 48		train 67.25%	loss 0.8868		val 71.88%	loss 0.8349
epoch 49		train 70.53%	loss 0.8246		val 73.21%	loss 0.8048
epoch 50		train 70.96%	loss 0.8213		val 70.03%	loss 0.8248
epoch 51		train 71.41%	loss 0.8097		val 66.58%	loss 0.9119
epoch 52		train 70.57%	loss 0.7899		val 73.21%	loss 0.7411
epoch 53		train 71.60%	loss 0.7852		val 72.68%	loss 0.7509
epoch 54		train 71.64%	loss 0.7795		val 67.11%	loss 0.9522
epoch 55		train 71.99%	loss 0.7695		val 71.88%	loss 0.7907
epoch 56		train 75.31%	loss 0.7143		val 73.74%	loss 0.7805
epoch 57		train 75.05%	loss 0.7145		val 74.80%	loss 0.7056
epoch 58		train 74.46%	loss 0.6939		val 74.27%	loss 0.7806
epoch 59		train 76.41%	loss 0.6774		val 76.92%	loss 0.6468
epoch 60		train 74.95%	loss 0.6902		val 77.19%	loss 0.6884
epoch 61		train 76.15%	loss 0.6717		val 74.54%	loss 0.7314
epoch 62		train 75.57%	loss 0.7038		val 75.86%	loss 0.6983
epoch 63		train 76.09%	loss 0.6550		val 77.72%	loss 0.6644
epoch 64		train 76.45%	loss 0.6491		val 75.86%	loss 0.6865
epoch 65		train 78.01%	loss 0.6260		val 80.90%	loss 0.5996
epoch 66		train 78.36%	loss 0.6253		val 79.31%	loss 0.6585
epoch 67		train 78.78%	loss 0.6161		val 79.05%	loss 0.6314
epoch 68		train 78.59%	loss 0.6122		val 75.60%	loss 0.6971
epoch 69		train 78.78%	loss 0.6141		val 80.11%	loss 0.6343
epoch 70		train 78.88%	loss 0.5957		val 78.25%	loss 0.6245
epoch 71		train 78.62%	loss 0.6105		val 76.66%	loss 0.6787
epoch 72		train 79.24%	loss 0.5964		val 78.25%	loss 0.6444
epoch 73		train 78.98%	loss 0.5902		val 79.58%	loss 0.6146
epoch 74		train 79.27%	loss 0.5874		val 79.05%	loss 0.6209
epoch 75		train 78.69%	loss 0.5987		val 77.98%	loss 0.6369
epoch 76		train 79.11%	loss 0.5857		val 77.19%	loss 0.6403
epoch 77		train 80.05%	loss 0.5754		val 79.58%	loss 0.6069
epoch 78		train 79.60%	loss 0.5729		val 76.66%	loss 0.6566
epoch 79		train 80.51%	loss 0.5656		val 78.25%	loss 0.6597

```
epoch 80 | train 81.12% loss 0.5582 | val 80.64% loss 0.6035  
(d21) pigeon@LAPTOP-GARN0974:~/ml-week/flower_identification/flower_identification$ python src/predict.py  
test_accuracy : 79.1860% test_loss : 0.6008
```

共训练80轮，最终测试集正确率为79.1860%。

注：由于模型权重大小超出github单个文件大小限制，因此复现需要在本地重新跑一遍train.py